A Big Data Framework for Price Tracking and Product Matching in Sri Lankan E-Commerce

P.K.K. Dias

April 28, 2026

Contents

Chapter 1

Introduction

Chapter 2

Literature Review

2.1 Crawling the web

Web crawling is the process of systematically browsing the web and extracting content from its pages. It is a fundamental technique used in various applications, most notably in search engines, where it is used to index the web and make it searchable. Another common use of web crawling is in data mining and web scraping, where it is used to collect data from websites for analysis or other purposes. More recently, web crawling has also been used in the context of training large language models, where it is used to gather vast amounts of text data from the web to train these models. Web crawlers, also known as spiders or bots, are automated programs that perform this task.

Crawlers typically traverse the web by following hyperlinks from one page to another, starting from a set of seed URLs [1]. They can be designed to crawl the entire web or to focus on specific domains, topics, or types of content.

A standard crawler operates in several stages [2]:

1. **Seed URLs**: The crawler starts with a list of initial URLs, known as seed URLs, which are the starting points for the crawling process.
2. **Downloading**: The crawler sends HTTP requests to the URLs and downloads the content of the web pages. The downloaded content is stored in local storage or cloud storage for further processing.

3. **Extracting links**: The crawler parses the downloaded pages to extract hyperlinks, which are then added to a crawl frontier.

A crawl frontier is a data structure that manages the list of URLs to be crawled, ensuring that the crawler does not revisit the same URL multiple times and that it adheres to any specified crawling policies (e.g., depth limits, domain restrictions). The crawl frontier can also be used as a queue to manage iterative and repeated crawling, allowing the crawler to revisit pages at regular intervals to check for updates, maintaining the freshness of the dataset [1].

Traditional web crawlers worked by sending HTTP requests to web servers and downloading the HTML content of web pages. However, such crawlers may struggle with modern websites that heavily rely on JavaScript and AJAX to render content dynamically via client-side rendering [3]. Modern web crawlers often need to use headless browsers or tools like Selenium, Puppeteer, or Playwright to handle JavaScript content and ensure that they can access the full content of web pages [4]. These tools allow the crawler to execute JavaScript and render the page as a regular browser would, while consuming fewer resources than a full browser, enabling the crawler to access dynamic content effectively.

It is also important to consider the legal and ethical implications of web crawling.

1. **Webmaster intent**: Webmasters may not want their content to be crawled or indexed, and they can use the `robots.txt` file to specify which parts of their site should not be accessed by crawlers [5].

2. **Server load**: Crawling can put a significant load on web servers, especially if the crawler is not designed to be respectful of the site's resources. It is important for crawlers to implement politeness policies, such as rate limiting, to avoid overwhelming servers [1].

3. **Data privacy**: Crawling and scraping can raise privacy concerns, especially if personal data is collected without consent. It is crucial to ensure that any data collected through crawling is handled in compliance with relevant data protection laws and regulations, such as the General Data Protection Regulation (GDPR) in the European Union [6].

A significant challenge in modern web crawling is the use of anti-bot measures by websites, such as CAPTCHAs, IP blocking, and rate limiting, which can hinder the crawling process [7]. Advanced security services such as Cloudflare employ fingerprinting techniques to identify and block automated traffic, making it increasingly difficult for crawlers to access content on certain websites [8]. Crawlers need to be designed to navigate these challenges while still adhering to ethical and legal guidelines Moreover, content hidden behind paywalls or login screens can also pose challenges for crawlers, as they may require authentication or subscription to access the content. Most crawlers, including Google's [9] generally do not access such content, unless explicit permission and credentials are provided by the site owner.

2.2 Boilerplate detection and removal

A webpage is a document that contains both content and presentation information. The content of a webpage is the information that is relevant to the user, while the presentation information is the information that is used to display the webpage. Boilerplate refers to the presentation information that is not relevant to the user, such as navigation menus, advertisements, and other non-content elements.

Boilerplate detection and removal is the process of identifying and removing boilerplate from a webpage so that only the relevant content remains. This is an important task for many applications, such as web scraping, information retrieval, and natural language processing as boilerplate/template content can interfere with the accuracy of these applications.

There are primarily two approaches to boilerplate detection and removal [10]:

1. **Site-Level Methods**: These methods analyze multiple pages from the same website to identify common patterns and structures that are likely to be boilerplate. By comparing the structure and content of different pages, site-level methods can effectively identify and remove boilerplate elements that are consistent across the site.

2. **Page-Level Methods**: These methods analyze each page individually to identify boilerplate elements based on features such as link density, text density, and the structure of the HTML document.

2.2.1 Site-level methods

Site Style Trees (SST)

Using the DOM from a sample of pages from a website, a site style tree (SST) can be constructed to represent the common structure of that specific website. For each unique subtree which appears in the SST, a count is maintained to track how many pages contain that subtree. Subtrees that appear in a large number of pages are likely to be boilerplate, while those that appear in fewer pages are more likely to be content. By analyzing the SST, it is possible to identify and remove boilerplate elements from the pages of the website, leaving only the relevant content. [11]

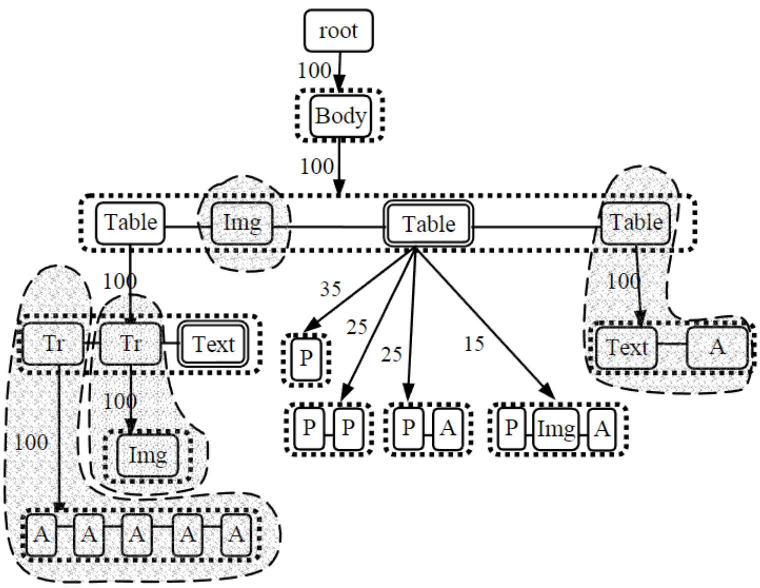

Figure 2.1: An example of site style trees (SST) [11]

Using subtree hashes

Similar to site style trees, subtree hashes can be used to identify boilerplate elements across multiple pages of a website. By computing a hash for each subtree in the DOM of a page, it is possible to compare these hashes across different pages to identify common subtrees that are likely to be boilerplate. Subtrees that have the same hash across multiple pages are likely to be boilerplate, while those that have unique hashes are more likely to be content. This method can be computationally efficient and effective in identifying boilerplate elements. [12]

2.2.2 Page-level methods

Body Text Extraction (BTE)

This is a heuristic-based method based on the observation that the main content of a webpage usually contains long streams of uninterrupted text with a low HTML tag density. By analyzing the structure of the HTML document and identifying areas with high text density and low tag density, it is possible to extract the main content of the page. While this is a simple method, one major drawback is that modern webpages are extremely complex and may not always follow this pattern, leading to inaccurate results. [13]

Shallow text features

This method first chunks the webpage into smaller sections based on the structure of the HTML document. It then computes various features for each chunk, such as link density (the ratio of links to text), text density (the ratio of text to HTML tags), and other structural features. High link density is generally indicative of boilerplate, while high text density is more indicative of content. [14]

This also builds on the concept of functional short text vs descriptive long text, where functional short text, characterized by short, incomplete sentences (e.g., home, contact us) is more likely to be boilerplate, while descriptive long text, characterized by full sentences and higher complexity is more likely to be content. [14]

Supervised machine learning techniques

While BTE and shallow text features can be effective in some cases, they may not always be sufficient to accurately identify boilerplate elements, especially on modern webpages with complex structures. Supervised machine learning techniques, such as neural networks and multi-layer perceptrons (MLPs), can be trained on labeled datasets of webpages to learn more complex patterns and features that distinguish boilerplate from content. By using a large and diverse training dataset, these models can learn to generalize well to new webpages and can be more effective in accurately identifying boilerplate elements. These techniques generally show a significant improvement in performance compared to heuristic-based methods, especially on modern webpages with complex structures. [15]

Two examples of supervised machine learning techniques for boilerplate detection and removal are Web2Text [13], which uses Convolutional Neural Networks (CNNs) to predict probabilities for chunks of a page, and the transition between blocks, and BoilerNet [16], which use bidirectional LSTMs that learn dependency between blocks and their neighbours.

2.2.3 Performance improvements

Removing boilerplate content from webpages have shown to significant performance gains in downstream tasks such as webpage classification and clustering, as it allows these tasks to focus on the relevant content of the page rather than being distracted by irrelevant boilerplate elements. In addition, it also reduces the amount of data that needs to be processed, which can lead to faster processing times and improved efficiency in these tasks.

Using site style trees (SSTs) to remove boilerplate has been shown to improve the accuracy in web page classification tasks from 0.625 to 0.954 [11]. Using the same method, the average F1-score for k-means clustering also improved from 0.506 to 0.751.

Bibliography

[1] M. Kumar, R. Bhatia, and D. Rattan, "A survey of web crawlers for information retrieval," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 7, no. 6, p. e1218, 2017.

[2] M. A. Kausar, V. Dhaka, and S. K. Singh, "Web crawler: a review," *International Journal of Computer Applications*, vol. 63, no. 2, pp. 31–36, 2013.

[3] A. Goel, J. Zhu, R. Netravali, and H. V. Madhyastha, "Sprinter: Speeding up High-Fidelity crawling of the modern web," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, (Santa Clara, CA), pp. 893–906, USENIX Association, Apr. 2024.

[4] A. Mesbah, A. Van Deursen, and S. Lenselink, "Crawling ajax-based web applications through dynamic analysis of user interface state changes," *ACM Transactions on the Web (TWEB)*, vol. 6, no. 1, pp. 1–30, 2012.

[5] Z. Chang, "A survey of modern crawler methods," in *Proceedings of the 6th international conference on control engineering and artificial intelligence*, pp. 21–28, 2022.

[6] M. A. Khder, "Web scraping or web crawling: State of art, techniques, approaches and application.," *International Journal of Advances in Soft Computing & Its Applications*, vol. 13, no. 3, 2021.

[7] K. Kothari, "Web crawler development: How to build a custom crawler," Mar 2026.

[8] Cloudflare, "Cloudflare bot management," 2026.

[9] Google, "Things to know about google's web crawling — google crawling infrastructure — crawling infrastructure — google for developers," 2026.

[10] J. Pomikálek, "Removing boilerplate and duplicate content from web corpora," *Disertacnı práce, Masarykova univerzita, Fakulta informatiky*, 2011.

[11] L. Yi, B. Liu, and X. Li, "Eliminating noisy information in web pages," in *ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD-2003)*, pp. 24–27.

[12] D. Gibson, K. Punera, and A. Tomkins, "The volume and evolution of web page templates," in *Special interest tracks and posters of the 14th international conference on World Wide Web*, pp. 830–839, 2005.

[13] T. Vogels, O.-E. Ganea, and C. Eickhoff, "Web2text: Deep structured boilerplate removal," in *European Conference on Information Retrieval*, pp. 167–179, Springer, 2018.

[14] C. Kohlschütter, P. Fankhauser, and W. Nejdl, "Boilerplate detection using shallow text features," in *Proceedings of the third ACM international conference on Web search and data mining*, pp. 441–450, 2010.

[15] R. Schäfer, "Accurate and efficient general-purpose boilerplate detection for crawled web corpora," *Language Resources and Evaluation*, vol. 51, no. 3, pp. 873–889, 2017.

[16] J. Leonhardt, A. Anand, and M. Khosla, "Boilerplate removal using a neural sequence labeling model," in *Companion Proceedings of the Web Conference 2020*, pp. 226–229, 2020.